

Проект: Мини-движок квеста на C#

Архитектура и дизайн (Этап 1)

Студент: [Ваше ФИО]
Группа: [Ваша Группа]

27 марта 2026 г.

Содержание

1	Введение и концепция	2
1.1	Цель проекта	2
1.2	Сюжетная завязка	2
2	Архитектура программы	2
2.1	Интерфейсы	2
2.2	Абстрактные классы	2
2.3	Основные классы (Ядро)	3
2.4	Команды	3
2.5	Условия (Conditions)	3
2.6	Эффекты (Effects)	3
2.7	Объекты взаимодействия	4
2.8	События (Events)	4
3	Структура мира (Локации)	4
4	Игровые механики	4
4.1	Механика 1: Тёмный коридор	4
4.2	Механика 2: Запертая дверь	4
4.3	Механика 3: Генератор	5
4.4	Механика 4: Одноразовая ловушка	5
5	Квесты	5
5.1	Квест 1: Включить генератор	5
5.2	Квест 2: Добраться до выхода	5
6	Принципы реализации (Согласно ТЗ)	5

1 Введение и концепция

1.1 Цель проекта

Разработка консольной текстовой игры-квеста с гибкой архитектурой, основанной на принципах объектно-ориентированного программирования: интерфейсы, абстрактные классы, наследование, полиморфизм, композиция.

1.2 Сюжетная завязка

Игрок приходит в себя в заброшенном исследовательском комплексе. Память частично утеряна.

- **Цель:** Восстановить питание комплекса и выбраться наружу.
- **Препятствия:** Запертые двери, темные коридоры, сломанное оборудование, ловушки.
- **Инструменты:** Предметы, найденные на складе и в локациях.

2 Архитектура программы

2.1 Интерфейсы

- `ICommand` — базовый интерфейс для команд пользователя. Метод: `Execute(GameState state)`
- `IInteractable` — для объектов взаимодействия. Метод: `Interact(Player player, GameState state)`
- `ICondition` — для проверки условий мира. Метод: `Check(GameState state)`
- `IEffect` — для изменения состояния мира. Метод: `Apply(GameState state)`

2.2 Абстрактные классы

- `CommandBase` — базовая реализация команд
- `ConditionBase` — базовый класс для условий
- `EffectBase` — базовый класс для эффектов
- `GameEventBase` — базовый класс для событий
- `InteractableObject` — базовый класс для объектов локации

2.3 Основные классы (Ядро)

- `Game` — управляет игровым циклом и парсингом ввода
- `GameState` — хранилище состояния (здоровье, инвентарь, флаги, журнал)
- `Player` — представление игрока
- `Location` — описание локации (название, описание, выходы, объекты, события)
- `Quest` — информация о задании
- `CommandParser` — разбирает строку ввода

2.4 Команды

- `HelpCommand` — вывод списка команд
- `LookCommand` — описание текущей локации
- `GoCommand` — переход между локациями
- `InteractCommand` — взаимодействие с объектом
- `InventoryCommand` — вывод инвентаря
- `StatusCommand` — статус игрока и квестов

2.5 Условия (Conditions)

- `HasItemCondition` — проверка наличия предмета
- `FlagCondition` — проверка флага мира
- `HealthCondition` — проверка здоровья
- `AndCondition`, `OrCondition`, `NotCondition` — составные условия

2.6 Эффекты (Effects)

- `AddItemEffect`, `RemoveItemEffect` — манипуляция инвентарем
- `SetFlagEffect` — установка флага
- `DamageEffect`, `HealEffect` — изменение здоровья
- `LogEffect` — запись в журнал
- `AddExitEffect` — открытие выхода
- `ChangeLocationEffect` — смена локации

2.7 Объекты взаимодействия

- **Chest** — контейнер с предметами
- **Door** — блокирует переход, требует условия
- **NPC** — терминал, выдает информацию
- **Trap** — ловушка, наносит урон

2.8 События (Events)

- **OnEnterLocationEvent** — при входе в локацию
- **OnTurnEvent** — каждый ход
- **OneTimeEvent** — одноразовое событие

3 Структура мира (Локации)

1. **Hall (Холл)** — Стартовая зона. Безопасно. Выходы: **Storage**, **DarkCorridor** (заперто).
2. **Storage (Склад)** — Захламленное помещение. Предметы: **Key**, **Torch**. Есть ловушка.
3. **DarkCorridor (Тёмный коридор)** — Без освещения наносит урон. Выходы: **Hall**, **GeneratorRoom**.
4. **GeneratorRoom (Комната генератора)** — Панель управления. Требуется **Fuse** и **Wrench**.
5. **Exit (Выход)** — Финальная локация. Открывается после включения генератора.

4 Игровые механики

4.1 Механика 1: Тёмный коридор

- **Событие:** `OnEnterLocationEvent` в **DarkCorridor**
- **Условие:** `NotCondition(HasItemCondition("Torch"))`
- **Эффект:** `DamageEffect(10) + LogEffect("Вы поранились в темноте")`

4.2 Механика 2: Запертая дверь

- **Объект:** **Door** между **Hall** и **DarkCorridor**
- **Условие:** `HasItemCondition("Key")`
- **Эффект:** `AddExitEffect("DarkCorridor")`

4.3 Механика 3: Генератор

- **Объект:** NPC (Терминал) в GeneratorRoom
- **Условие:** `AndCondition(HasItemCondition("Fuse"), HasItemCondition("Wrench"))`
- **Эффект:** `SetFlagEffect("GeneratorOn" + AddExitEffect("Exit"`

4.4 Механика 4: Одноразовая ловушка

- **Объект:** Trap в Storage
- **Событие:** OneTimeEvent
- **Эффект:** `DamageEffect(20) + LogEffect("Ловушка сработала"`

5 Квесты

5.1 Квест 1: Включить генератор

- **Описание:** Найти Fuse и Wrench, активировать терминал
- **Условие завершения:** Флаг `GeneratorOn = true`
- **Награда:** Открывается доступ к Exit

5.2 Квест 2: Добраться до выхода

- **Описание:** Покинуть комплекс через шлюз
- **Условие завершения:** Игрок в локации Exit
- **Награда:** Победа в игре

6 Принципы реализации (Согласно ТЗ)

1. **Отсутствие логики в Main:** Метод Main только иницирует Game и запускает цикл
2. **Полиморфизм вместо проверок типов:** Запрещено `is`, `as`, `switch` по типам
3. **Единое состояние:** GameState — единственный источник правды
4. **Расширяемость:** Новые классы без изменения существующего кода